

NanoTable

Dynamic Data Management System

Database Systems (DBS)

BY:

Ayesha Tariq 24K-0785

Sanabil Mustafa 24K-1014

1. Executive Summary

NanoTable is a web-based dynamic data management system inspired by Notion's table feature. It allows users to create fully custom tables — defining their own columns with specific data types — and manage structured data entirely through a browser-based interface. Unlike rigid single-purpose tools such as separate habit trackers or expense managers, NanoTable provides a unified, flexible workspace where the same system can serve any structured data need a user has.

The project was built as a full-stack application using Python (Flask) for the backend, PostgreSQL for the database, and HTML/CSS/JavaScript for the frontend. The core database design uses a meta-driven schema — a database that describes and stores other database structures — which is the central academic contribution of this project.

2. Introduction

2.1 Problem Statement

Existing data management tools such as habit trackers, expense managers, reading logs, and attendance systems are built with fixed schemas. Each is a separate application designed for one specific purpose. Users who need to track multiple types of data must switch between multiple tools, cannot reuse data across contexts, and have no flexibility to define what fields matter to them.

2.2 Proposed Solution

NanoTable solves this by providing a single application where users define their own table structures at runtime. A user creates a table (e.g., "Expense Tracker"), defines columns (e.g., Amount as NUMBER, Category as TEXT, Date as DATE), and inserts rows of data. The system stores both the schema and the data dynamically using a five-table relational design, without ever creating new SQL tables at runtime.

2.3 Project Scope

The system covers user authentication, full CRUD operations on user-defined tables, type-aware filtering with SQL-level WHERE clauses, client-side sorting and search, an analytics view with automated number aggregation and monthly date grouping, and CSV export. The system is deployed locally via Flask on port 5000.

3. System Architecture

3.1 Three-Tier Architecture

The application follows a standard three-tier architecture:

Tier	Technology	Responsibility
Presentation Layer	HTML, CSS, JavaScript	User interface, form inputs, data display, analytics charts
Application Layer	Python 3 + Flask	Business logic, API routing, request handling, SQL query construction
Database Layer	PostgreSQL	Persistent storage of users, table schemas, rows, and cell values

3.2 Request Flow

When a user interacts with the frontend (e.g., clicks 'Apply Filter'), the following sequence occurs:

1. JavaScript constructs the API URL with query parameters (e.g., GET /tables/3/data?column=Amount&op=>=&value=500)
2. Flask receives the request and parses the parameters
3. Flask constructs a parameterised SQL query with a subquery WHERE clause
4. PostgreSQL executes the query and returns only matching rows
5. Flask formats the result as JSON and sends it back
6. JavaScript receives the JSON and re-renders the table in the browser

4. Database Design

4.1 Design Philosophy — Meta-Driven Schema

Key Insight: Instead of creating a new SQL table every time a user creates a 'table', NanoTable stores the schema of user-defined tables inside the database itself. This is a meta-driven or entity-attribute-value (EAV) pattern.

This approach means the database schema never changes at runtime. All user data — regardless of which table it belongs to or how many columns it has — is stored in the same five tables.

4.2 Schema — Five Core Tables

Users

Stores registered user accounts.

Column	Type	Constraint	Description
user_id	SERIAL	PRIMARY KEY	Auto-incrementing unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for login
password	VARCHAR(255)	NOT NULL	Bcrypt-hashed password

Tables

Stores each user-created table (e.g., 'Expense Tracker', 'Reading Log').

Column	Type	Constraint	Description
table_id	SERIAL	PRIMARY KEY	Unique table identifier
user_id	INT	FK → Users(user_id) ON DELETE CASCADE	Owner of this table
table_name	VARCHAR(100)	NOT NULL	Display name of the table
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

Columns

Stores the column schema for each user-created table. This is the core of the meta-driven design.

Column	Type	Constraint	Description
column_id	SERIAL	PRIMARY KEY	Unique column identifier
table_id	INT	FK → Tables(table_id) CASCADE	Which table this column belongs to
column_name	VARCHAR(100)	NOT NULL	Display name of the column
data_type	ENUM	NOT NULL	One of: TEXT, NUMBER, DATE, BOOLEAN
column_order	INT		Display order of column in UI

Rows

Represents a single row of data in a user-created table. Contains only metadata — actual values are stored in CellValues.

Column	Type	Constraint	Description
row_id	SERIAL	PRIMARY KEY	Unique row identifier
table_id	INT	FK → Tables(table_id) CASCADE	Which table this row belongs to
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp

CellValues

Stores the actual data for every cell. Each row stores one value per column. The correct value column is used based on the column's data type — the UNIQUE constraint ensures only one value exists per (row, column) pair.

Column	Type	Constraint	Description
cell_id	SERIAL	PRIMARY KEY	Unique cell identifier
row_id	INT	FK → Rows(row_id) CASCADE	The row this cell belongs to
column_id	INT	FK → Columns(column_id) CASCADE	The column this cell belongs to
value_text	TEXT	nullable	Used when column type = TEXT
value_number	DOUBLE	nullable	Used when column type = NUMBER
value_date	DATE	nullable	Used when column type = DATE
value_boolean	BOOLEAN	nullable	Used when column type = BOOLEAN

A CHECK constraint ensures exactly one value column is non-null per cell, enforcing data integrity at the database level.

4.3 Relationships

- Users → Tables: One-to-Many (one user can have many tables)
- Tables → Columns: One-to-Many (one table has many columns defining its schema)
- Tables → Rows: One-to-Many (one table has many rows of data)

- Rows + Columns → CellValues: Many-to-Many resolved (each cell is one row-column intersection)
- ON DELETE CASCADE is used throughout so deleting a table removes all its columns, rows, and cell values automatically

4.4 Indexes

Five indexes were created to optimise query performance on frequently-used foreign key lookups:

Index Name	Table	Column	Purpose
idx_tables_user_id	tables	user_id	Fast lookup of tables by user
idx_columns_table_id	columns	table_id	Fast schema lookup when loading a table
idx_rows_table_id	rows	table_id	Fast row retrieval for a given table
idx_cellvalues_row_id	cellvalues	row_id	Fast cell retrieval for a given row
idx_cellvalues_column_id	cellvalues	column_id	Fast cell retrieval for a given column

5. Backend (Flask API)

5.1 Technology Stack

Library	Version	Purpose
Flask	3.x	Web framework — handles HTTP routing and response formatting
psycopg2	2.9.x	PostgreSQL adapter — executes parameterised SQL queries
Werkzeug	3.x	Password hashing (generate_password_hash, check_password_hash)

5.2 API Endpoints

Method	Endpoint	Description
POST	/auth/register	Register a new user (hashes password with bcrypt)
POST	/auth/login	Authenticate user, return user_id on success
POST	/tables	Create a new user-defined table
GET	/tables?user_id=X	Get all tables belonging to a user
PUT	/tables/{id}	Rename a table
DELETE	/tables/{id}	Delete table + all its columns, rows, cells (CASCADE)

Method	Endpoint	Description
POST	/columns	Add a column to a table with name and data type
GET	/tables/{id}/columns	Get all columns for a table in order
DELETE	/columns/{id}	Delete a column and all its cell data (CASCADE)
PUT	/columns/reorder	Reorder columns by updating column_order
POST	/rows	Create an empty row
DELETE	/rows/{id}	Delete a row and all its cell values (CASCADE)
POST	/rows/full	Insert a complete row with all cell values at once
POST	/cells	Upsert a single cell value (ON CONFLICT UPDATE)
GET	/tables/{id}/data	Get full table data with optional SQL-level filtering

5.3 SQL-Level Filtering

The GET /tables/{id}/data endpoint supports type-aware filtering pushed down to PostgreSQL via a subquery. The frontend passes three query parameters: column (column name), value (the comparison value), and op (operator: =, !=, >, <, >=, <=, or contains).

The backend uses COALESCE across all four value columns cast to text to handle any data type in a single comparison expression, avoiding the need for separate branches for each type. The ILIKE operator is used for the 'contains' operation, providing case-insensitive text search at the database level.

6. Frontend

6.1 File Structure

File	Location	Responsibility
index.html	templates/	HTML structure — all screens, modals, and containers
style.css	static/	All visual styling, dark theme, animations, responsive layout
script.js	static/	All application logic — API calls, state management, rendering

6.2 State Management

The frontend maintains application state in JavaScript variables. A session persistence mechanism uses sessionStorage to preserve the logged-in user across browser tab switches without requiring

re-authentication. sessionStorage (not localStorage) was chosen so the session automatically clears when the browser is fully closed, balancing usability with security.

6.3 Key Features

- User authentication with register/login, session persistence across tab switches
- Sidebar with collapsible panel, table search, and pin-to-top functionality
- Per-table emoji icons stored in localStorage
- Column type badges (TEXT, NUMBER, DATE, BOOLEAN) with matching colour coding
- Client-side column sorting (ascending/descending) with active indicator
- Server-side filtering with 7 operator types (=, !=, >, <, >=, <=, contains)
- In-table text search (client-side, no extra API call)
- Row duplication using the existing POST /rows/full endpoint
- CSV export with timestamp in filename
- Analytics view: auto-detects NUMBER columns for sum/avg/min/max cards, DATE columns for monthly bar charts and breakdown tables
- Skeleton loading animation while table data loads
- Toast notification system (success, error, info, warning)
- Display row index (#1, #2, #3...) separate from database row_id to avoid numbering gaps after deletion

7. Analytics Feature

7.1 Overview

The Analytics tab appears on every table and generates reports automatically based on the columns present. It requires no configuration for basic use — the system detects column types and generates appropriate visualisations.

7.2 What It Generates

Column Type Detected	Analytics Generated
NUMBER	Summary cards showing: Total Sum, Average, Minimum, Maximum for each numeric column
DATE	Monthly bar chart showing data distribution across months, plus a breakdown table with count/total/avg/min/max per month
BOOLEAN	True vs False count with completion percentage (useful for habit trackers)
DATE + NUMBER	Monthly totals chart — groups NUMBER values by the month of the DATE column

The analytics are computed entirely from the data already loaded in memory from `GET /tables/{id}/data` — no additional backend endpoints were needed. This avoids extra database queries and keeps the feature self-contained in the frontend

8. Conclusion

NanoTable successfully demonstrates core database systems concepts in a working application. The meta-driven schema design — storing table structures as data rather than as actual SQL tables — is the central academic contribution, showing how a flexible schema-on-the-fly system can be built using standard relational database features.

The project covers all DBS course requirements: ER modelling, normalisation (the schema is in 3NF), SQL joins, foreign key constraints with CASCADE, indexing for performance, parameterised queries, and type-aware filtering using SQL WHERE clauses. The additional analytics feature demonstrates GROUP BY-style aggregation performed programmatically from query results.

Future improvements could include user-defined column constraints (NOT NULL, UNIQUE), table sharing between users, and a drag-to-reorder interface for columns.